

Aether — Kubernetes & Helm

Production-Grade Kubernetes Deployment

Aether generates complete Kubernetes manifests from a single YAML spec
— Pods, Services, Ingress, PVCs, ConfigMaps, Secrets, and HPA — then
deploys them via the native kube API.

Pod • Service • Ingress • PVC • ConfigMap • HPA

Helm Chart **Overview**

Aether ships a Helm chart for installing the control plane itself into any Kubernetes cluster. The values.yaml file controls every aspect of the deployment.

values.yaml Structure

```
# values.yaml - Aether Helm chart configuration
replicaCount: 1

image:
  repository: ghcr.io/aether/aether
  tag: "latest"
  pullPolicy: IfNotPresent

service:
  type: ClusterIP
  port: 5090          # Web dashboard port

ingress:
  enabled: true
  className: nginx
  hosts:
    - host: aether.example.com
      paths:
        - path: /
          pathType: Prefix
  tls:
    - secretName: aether-tls
      hosts:
        - aether.example.com

resources:
  requests:
    cpu: 250m
    memory: 256Mi
  limits:
    cpu: "1"
    memory: 512Mi

namespace:
  create: true
  name: aether-system

rbac:
  create: true          # ServiceAccount, Role, RoleBinding
  clusterWide: false    # Set true for cross-namespace management

monitoring:
  serviceMonitor:
    enabled: true        # Prometheus ServiceMonitor CRD
    interval: 30s
```

Install with Helm

```
# Add the Aether Helm repo and install
$ helm repo add aether https://charts.aether.dev
$ helm install aether aether/aether \
  --namespace aether-system --create-namespace \
  --values values.yaml

# Upgrade an existing installation
$ helm upgrade aether aether/aether \
  --namespace aether-system --values values.yaml
```

Resource Generation

Aether translates your workload spec into native Kubernetes objects. Every resource is generated with proper labels, annotations, and ownership references.



Pod

Container spec with resource limits, health probes, environment variables, and volume mounts. Labels include `app` and `managed-by: aether`.



Service

ClusterIP, NodePort, or LoadBalancer service generated from the `network.ports` and `network.service_type` spec fields.



Ingress

Ingress resource with host rules, path routing, and TLS termination. Supports `nginx`, `traefik`, and any `IngressClass`.



PVC

PersistentVolumeClaim with configurable access modes (`ReadWriteOnce`, `ReadOnlyMany`, `ReadWriteMany`) and storage class.



ConfigMap & Secret

ConfigMaps and Secrets created from the config spec. Can be mounted as volumes or injected as environment variables via `envFrom`.



HPA

HorizontalPodAutoscaler with CPU and memory targets. Generated from the scaling spec with `min/max replicas` and `target utilization`.

Generated Manifest Preview

```
# Preview what Aether will create without applying
$ aether run workload.yaml --dry-run --output yaml

# Output shows every K8s resource that would be created:
#   - Pod/Deployment with resource limits and probes
#   - Service (ClusterIP) on port 8080
#   - Ingress with TLS via cert-manager
#   - PVC (10Gi, ReadWriteOnce)
#   - ConfigMap (app-config) mounted at /etc/config
#   - HPA (2-10 replicas, 70% CPU target)
```

Namespace Management

Aether supports three ways to control the target namespace, with a clear precedence order.



CLI Flag (Highest Priority)

```
# Override namespace for any command
$ aether run workload.yaml \
  --namespace production

# Short form
$ aether run workload.yaml -n staging

# Works with all commands
$ aether status -n monitoring
$ aether logs my-app -n production
$ aether destroy my-app -n staging
```



Environment Variable

```
# Set namespace via environment variable
$ export AETHER_NAMESPACE=production

# All subsequent commands use this namespace
$ aether run workload.yaml
# Deploys to "production" namespace

# CLI flag still takes precedence
$ aether run workload.yaml -n staging
# Deploys to "staging" despite env var

# Useful in CI/CD pipelines
env:
  AETHER_NAMESPACE: ${ github.ref_name }
```

Precedence Order

--namespace / -n flag



AETHER_NAMESPACE env



Default: "default"

Advanced: Autoscaling, Ingress TLS & Volume Mounts

Autoscaling Spec

```
scaling:
  min_replicas: 2
  max_replicas: 10
  target_cpu_utilization: 70
  target_memory_utilization: 80
```

Generates an HPA with CPU and memory metric targets. Aether uses the `autoscaling/v2` API for multi-metric scaling.

Ingress with TLS

```
ingress:
  host: api.example.com
  path: /
  tls_secret: api-tls-cert
  annotations:
    cert-manager.io/cluster-issuer: letsencrypt
```

Generates an Ingress resource with TLS termination. Works with cert-manager for automatic certificate provisioning.

Monitoring in **Kubernetes**

Aether integrates with the Kubernetes ecosystem for monitoring, from kubectl to Prometheus.

kubectl Integration

```
# Aether labels all resources for easy filtering
$ kubectl get pods -l managed-by=aether

NAME           READY   STATUS    AGE
my-api-1       1/1     Running   2h
my-web-app-1   1/1     Running   5h

# Check HPA status
$ kubectl get hpa -l managed-by=aether

NAME      REFERENCE   TARGETS   MINPODS   MAXPODS
my-api    Pod/my-api  45%/70%   2         10

# View Aether-managed ingress
$ kubectl get ingress -l managed-by=aether

NAME      CLASS   HOSTS             ADDRESS
my-api    nginx   api.example.com   10.0.0.5
```

Aether Native Commands

```
# Unified status across all K8s workloads
$ aether status -n production

NAME      RUNTIME  STATE    READY  RESTARTS
my-api    kube     Running  true   0
my-web    kube     Running  true   0

# Stream logs from Kubernetes pods
$ aether logs my-api --follow -n production
[2026-04-12 10:05:12] Server started on :3000
[2026-04-12 10:05:13] Connected to database

# Health history and uptime tracking
$ aether health my-api
Uptime: 99.97% (720 checks)
Last state: Running | Restarts: 0
```

Prometheus ServiceMonitor

Automatic ServiceMonitor Generation

When monitoring is enabled in the Helm values, Aether creates a Prometheus ServiceMonitor CRD that automatically discovers and scrapes metrics from all Aether-managed workloads.

```
apiVersion: monitoring.coreos.com/v1
kind: ServiceMonitor
metadata:
  name: aether-workloads
  labels:
    managed-by: aether
spec:
  selector:
    matchLabels:
      managed-by: aether
  endpoints:
    - port: metrics
      interval: 30s
      path: /metrics
```

Deploy to **Kubernetes** Today

From a single workload spec to a full production Kubernetes deployment in one command.

Quick Start

```
# 1. Write your workload spec
$ aether template rest-api --name my-api

# 2. Deploy to Kubernetes
$ aether run workload.yaml -n production
Creating ConfigMap: my-api-config ... done
Creating PVC: my-api-data (10Gi) ... done
Creating Pod: my-api ... done
Creating Service: my-api (ClusterIP:8080) ... done
Creating Ingress: my-api (api.example.com) ... done
Creating HPA: my-api (2-10 replicas) ... done

Workload my-api deployed to kubernetes (production)

# 3. Verify deployment
$ aether status my-api -n production
NAME      RUNTIME  STATE    READY  RESTARTS  AGE
my-api    kube     Running  true   0         30s
```

Full Resource Control

6

K8s Resource Types

3

Service Types

v2

HPA API Version

3

Namespace Sources

Kubernetes Without the YAML Sprawl

Aether generates production-ready Kubernetes manifests from a concise workload spec. Pod, Service, Ingress, PVC, ConfigMap, and HPA — all from one file, deployed with one command.